

Sistema de Soporte Técnico ISP

Entrega 3 · Aplicaciones Distribuidas (ISR-701) · Equipo ACC

Carlos Carpio — Datos/Reportes · expone la demo

Cristhian Pacheco — Ticket-service / Spark

Robinson Cando — Pruebas / Métricas / Protocolo

Jeremy Álvarez — Arquitectura / Notificaciones / IA

Código: PFC-E3 · ISR-701 · Repositorio: <https://github.com/carlospatroner-boop/PFC-SOPORTE-ISP>

Julio 2026

Objetivo de hoy y hoja de ruta

Objetivo: demostrar la capa de datos distribuida y el pipeline paralelo de la E3.

- Arquitectura vigente y qué cambió en E3
- Decisiones técnicas: fragmentación, CAP/PACELC
- Evidencia real: pruebas, métricas y pipeline Spark
- Demo en vivo: crear ticket → notificación
- Riesgos, plan hacia E4 y conclusiones

El problema: soporte técnico de un ISP

- Dominio: gestión de tickets de soporte técnico de Internet
- Usuarios: cliente, técnico de zona, administrador
- RNF crítico: SLA de resolución por prioridad
- RNF crítico: consistencia bajo escrituras concurrentes
- RNF crítico: escalar a más de 500k incidencias históricas

Arquitectura de contenedores (C4 Nivel 2)

Cliente · Técnico · Administrador

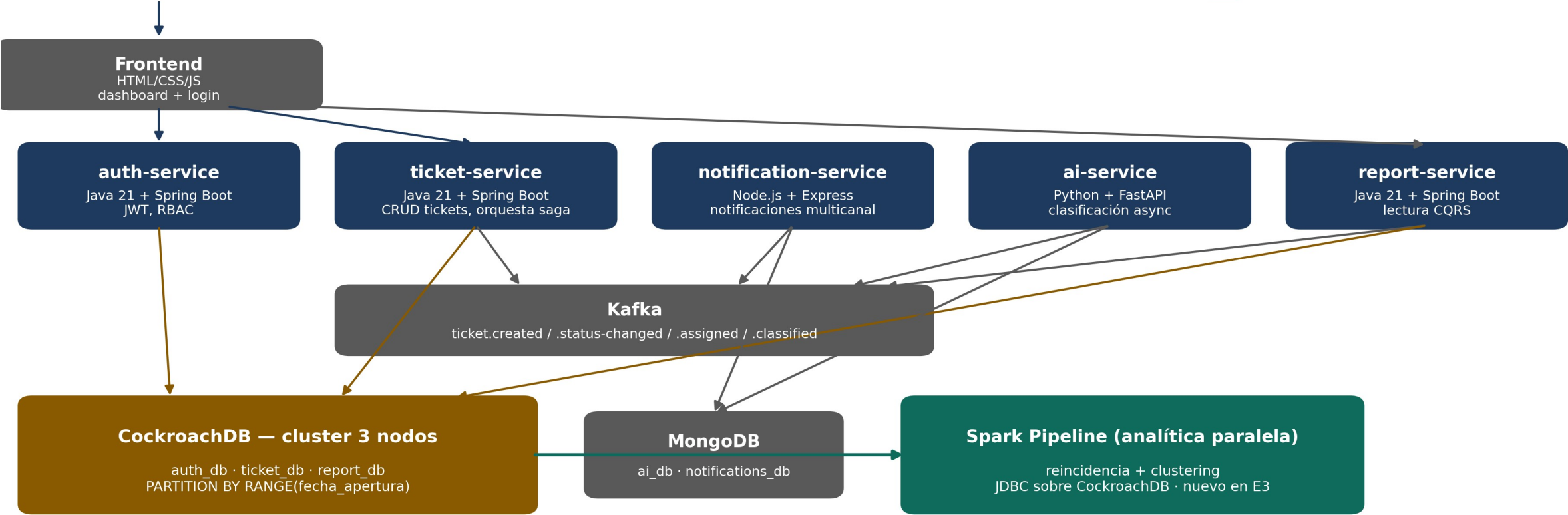


Diagrama propio, notación C4 · ámbar = fragmentado en E3 · teal = nuevo en E3

Qué cambió respecto a E2

- tickets: fragmentado por fecha_apertura (antes: por zona)
- Clúster CockroachDB real de 3 nodos (antes: mono-nodo)
- +3 métricas Prometheus incrementales en ticket-service
- +Testcontainers: pruebas contra CockroachDB real
- +Pipeline Spark: nuevo componente analítico paralelo

Lo planificado vs. lo construido

- ✓ Fragmentación por fecha + ADR-0003 — hecho, verificado
- ✓ Testcontainers + 3 métricas Prometheus — hecho, 0 advertencias
- ✓ Pipeline Spark, 5 transformaciones, 600k filas — hecho
- Video de tolerancia a fallos (1 y 2 nodos) — pendiente
- Protocolo estadístico, 50 corridas, IC 95% — pendiente

Decisión: fragmentación por fecha_apertura

```
CREATE TABLE tickets (  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  id UUID NOT NULL DEFAULT gen_random_uuid(),  
  ...  
  PRIMARY KEY (created_at, id)  
) PARTITION BY RANGE (created_at) (  
  PARTITION tickets_2026_q1  
    VALUES FROM (MINVALUE) TO ('2026-04-01'),  
  ...  
);
```

Alternativa descartada: PARTITION BY LIST(zona).

Criterio: exigencia explícita de la guía de E3 para el equipo ACC [1].

Ubicación CAP / PACELC del sistema

- CockroachDB = sistema CP: prioriza consistencia [2]
- Ante partición: rechaza escrituras conflictivas, no arriesga
- Consenso Raft: quórum 2 de 3, tolera 1 nodo caído [4]
- Verificado en vivo: conflicto real → SQLSTATE 40001

Artefactos verificables en el repositorio

- db-cluster/: 3 nodos, esquema fragmentado, ADR-0003
- services/svc-principal: refactor completo + Testcontainers
- spark/: pipeline + baseline + protocolo (600.000 filas)
- tests/integration: 2 pruebas contra CockroachDB real

Evidencia en vivo: métricas Prometheus reales

```
$ curl localhost:8002/actuator/prometheus
```

```
# HELP crdb_pool_active_connections Conexiones activas del pool
# TYPE crdb_pool_active_connections gauge
crdb_pool_active_connections{application="ticket-service"} 0.0

# HELP crdb_transaction_retries_total Reintentos por conflicto
# TYPE crdb_transaction_retries_total counter
crdb_transaction_retries_total{application="ticket-service"} 0.0
```

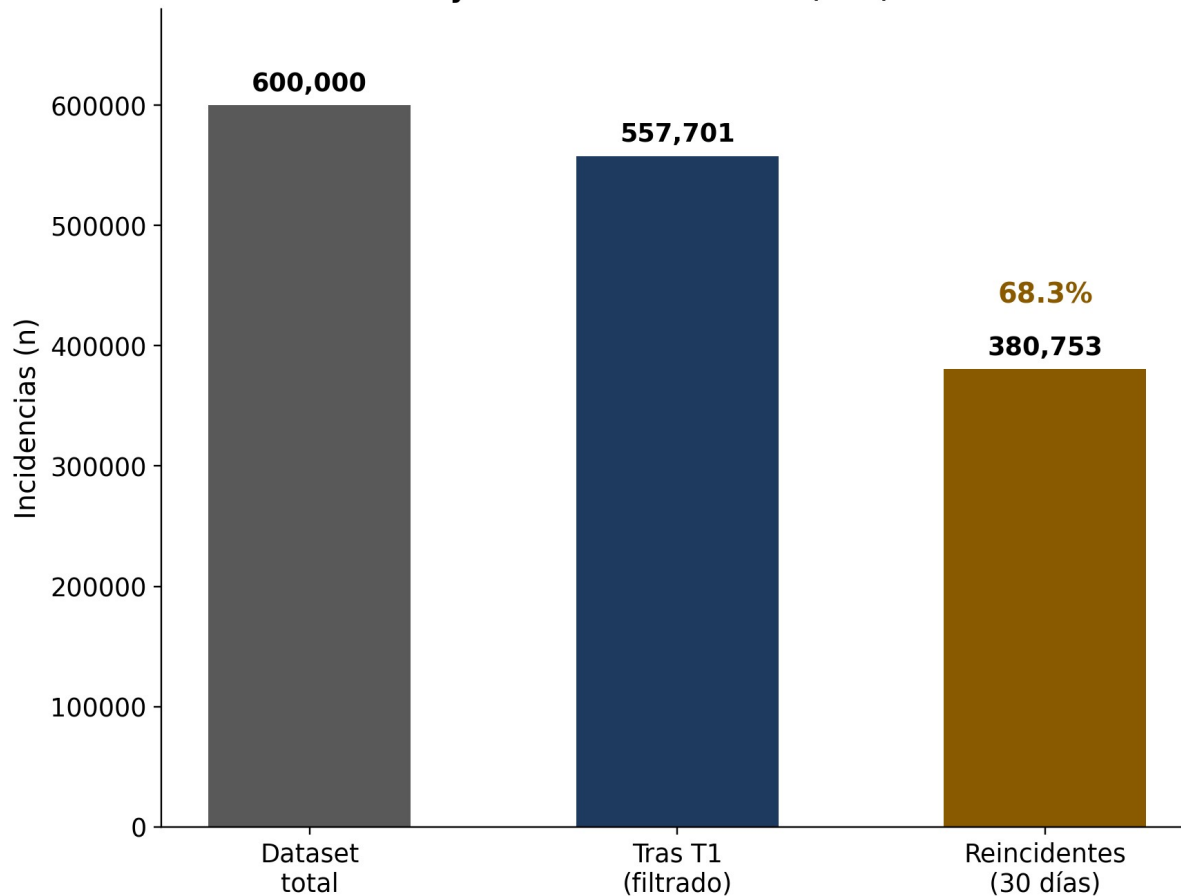
```
$ promtool check metrics < salida.prom → 0 advertencias
```

Verificación de corrección (medida, no supuesta)

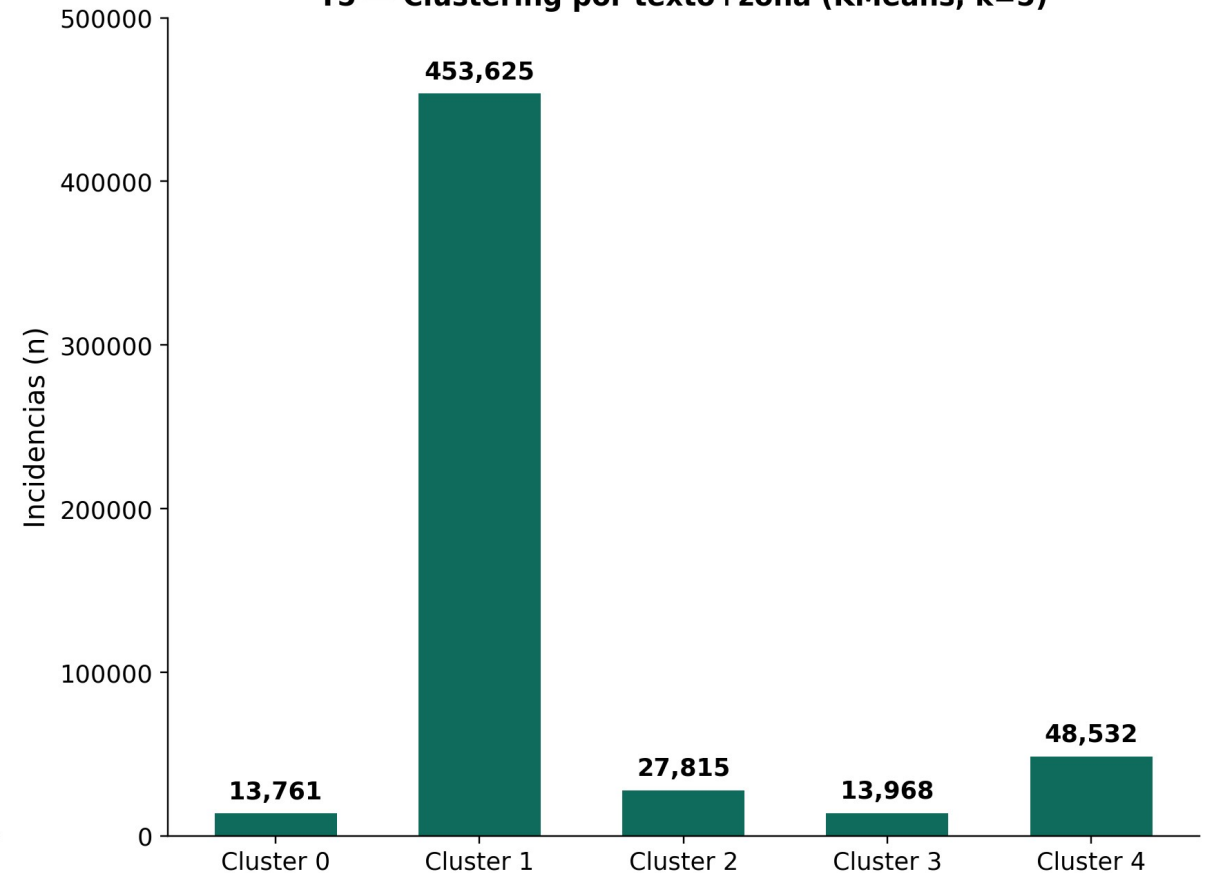
- ✓ Aislamiento serializable: conflicto real → SQLSTATE 40001
- ✓ 3 métricas Prometheus: 0 advertencias de promtool
- ✓ Testcontainers: 2 de 2 pruebas en verde, CockroachDB real

Pipeline Spark sobre 600.000 incidencias

Filtrado y reincidencia — n=600,000, seed=42



T5 — Clustering por texto+zona (KMeans, k=5)



Ejecución única, determinista (seed=42) · spark_pipeline.ipynb, 5 transformaciones [5]

Demo: crear ticket → Kafka → notificación

- Objetivo: mostrar la integración asíncrona real entre servicios
- Paso 1: POST /tickets con JWT de un cliente autenticado
- Paso 2: ticket-service publica ticket.created en Kafka
- Paso 3: notification-service consume y genera la notificación
- Resultado esperado: notificación visible en menos de 2 segundos

Resultado observado (respaldo de contingencia)

```
bash - demo E3 - ticket-service + notification-service

$ curl -X POST localhost:8002/api/v1/tickets -H "Authorization: Bearer $TOKEN" \
  -d '{"zone":"QUEVEDO_NORTE","title":"Sin acceso a Internet", ... }'

HTTP 201 Created
{"data":{"ticketId":"2129d338-848c-4145-a870-5aeba2764384",
  "zone":"QUEVEDO_NORTE","status":"NUEVO",
  "createdAt":"2026-07-26T17:47:25.79-05:00"},
  "message":"Ticket creado exitosamente"}

# ticket-service publica el evento ticket.created en Kafka
# notification-service lo consume automaticamente

$ curl localhost:8003/api/v1/notifications?ticketId=2129d338-...

HTTP 200 OK
{"data":[{"ticketId":"2129d338-848c-4145-a870-5aeba2764384",
  "eventType":"ticket.created","channel":"EMAIL",
  "message":"Recibimos tu solicitud (ticket 2129d338...).
    Te avisaremos cuando haya novedades.",
  "createdAt":"2026-07-26T22:47:27.19Z"}], "message":"OK"}
```

Ejecutado en vivo el 26/07/2026 · ticket 2129d338-... · valida el objetivo del Paso 12

Riesgos, deuda técnica y plan hacia E4

- Riesgo: zone cruza 4 particiones — mitigado con índice secundario
- Deuda técnica: sin API Gateway único — planificado para E4
- Pendiente: video de tolerancia a fallos + protocolo estadístico
- E4 reutiliza sin modificar: clúster, 3 métricas, pipeline Spark

Conclusiones del equipo

- Fragmentación exigida $\neq$ patrón de acceso real: documentar el costo, no forzarlo
- Verificación empírica (Testcontainers, promtool) halló un problema real antes que un usuario
- Pipeline con objetivo propio del dominio $>$ transformaciones genéricas

Declaración de uso de IA generativa

- Herramienta: Claude Code (Anthropic), asistente conversacional
- Propósito: scaffolding, implementación guiada, depuración y pruebas
- Alcance: todas las diapositivas — contenido revisado y verificado
- Decisiones de arquitectura y verificación funcional: siempre del equipo

Referencias

- [1] M. T. Özsu and P. Valduriez, Principles of Distributed Database Systems, 4th ed. Springer, 2020.
- [2] E. Brewer, "CAP twelve years later," IEEE Computer, vol. 45, no. 2, pp. 23–29, 2012.
- [3] D. J. Abadi, "Consistency tradeoffs...", IEEE Computer, vol. 45, no. 2, pp. 37–42, 2012.
- [4] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," USENIX ATC, 2014.
- [5] M. Zaharia et al., "Resilient Distributed Datasets," NSDI, 2012.